МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ “ЛЬВІВСЬКА ПОЛІТЕХНІКА”
ІНСТИТУТ КОМП’ЮТЕРНИХ НАУК ТА ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ
кафедра систем штучного інтелекту



ЗВІТ

про виконання лабораторної роботи № 2

з курсу ***«Методологія та процеси моделювання у штучному інтелекті»***

Виконав:
ст. групи КНСШ-13
Дуда Артем Сергійович
Перевірив:
проф. каф. СШІ, д.т.н.,
с.н.с.,
Косаревич Р. Я.

ЛЬВІВ – 2026

Зміст

1. Завдання.....3
2. Хід роботи.....4
2.1 Опис набору даних та методів обробки даних для сценарії А, Б, В....4
2.2 Порівняння та аналіз результатів моделей.....5
2.3 Аналіз впливу SMOTE та ризиків перенавчання.....7
4. Висновок.....8
4. Список використаних джерел.....9
Додаток А.....10

Тема: Порівняння традиційних ML-алгоритмів для задачі класифікації/регресії.

Мета: Ознайомитись з основними ML-алгоритмами та навчитися обирати й порівнювати їхню ефективність для конкретної задачі.

1. Завдання

1. Використати підготовлений у ЛР №1 датасет.

2. Навчити 2-3 різні моделі (наприклад, Логістична регресія, SVM, Дерево рішень, Випадковий ліс, Градієнтний бустинг) для вирішення задачі класифікації/регресії.

3. Провести порівняльні експерименти на кількох сценаріях даних:

- **3.1.** Сценарій А: Базові дані (мінімальне очищення).

- **3.2.** Сценарій Б: Повноцінно очищені дані (просунута імп'ютація + кодування + трансформація).

- **3.3.** Сценарій В: Очищені дані + застосування методів аугментації (SMOTE/ADASYN).

4. Оцінити продуктивність кожної моделі за відповідними метриками (Accuracy, Precision, Recall, F1-Score, ROC-AUC для класифікації; MSE, MAE, R2 для регресії).

5. Проаналізувати сильні та слабкі сторони кожного алгоритму.

6. Сформувати підсумковий звіт про стан даних та виконані перетворення.

Звіт повинен містити:

- Порівняльну таблицю результатів моделювання на різних етапах підготовки даних.

- Детальний аналіз того, чи покращила аугментація якість передбачення і за яких умов вона може викликати перенавчання (overfitting).

Кожне завдання можна реалізувати з використанням популярних бібліотек (наприклад, Scikit-learn, TensorFlow, PyTorch) та мови програмування Python.

2. Хід роботи

2.1 Опис набору даних та методів обробки даних для сценарії А, Б, В

Початковий датасет містить 8 500 записів та 18 ознак, які описують звички користувачів перед сном, рівень фізичної активності, вживання кофеїну та показники сну. Цільова змінна sleep_debt_category містить 4 класи з вираженим дисбалансом:

- Moderate Debt: 52.5%
- Mild Deficit: 23.6%
- Optimal Recovery: 16.3%
- Severe Sleep Debt: 7.6%

У попередній лабораторній було виконано обробку даних, а в цій лабораторній кінцеві дані застосовані у сценарії В, як зазначено в завданні. У сценаріях А і Б частково повторено перші етапи попередньої обробки з лабораторної роботи №1. Було виконано наступні операції в даній лабораторній, переважно для сценаріїв А і Б:

1. Імп'ютація пропусків:

- Сценарій А: Застосовано базову медіанну імп'ютацію.
- Сценарій Б та В: Застосовано IterativeImputer для відновлення відсутніх значень з урахуванням взаємозв'язків між ознаками.

2. Інженерія ознак:

- Створено ознаки, що описують сукупне навантаження перед сном.
- Додано поліноміальні ознаки.

3. Масштабування та трансформація: нормалізовано за допомогою PowerTransformer та стандартизовано значення.

4. Аугментація даних: У Сценарії В застосовано синтетичне збалансування класів SMOTE виключно до навчальної вибірки (X_train), що дозволило вирівняти частки всіх 4 класів до 25% без витоку даних у тестову вибірку.

2.2 Порівняння та аналіз результатів моделей

Було натреновано моделі та отримано результати у таблиці-порівнянні Табл. 2.1. Код можна знайти у Додатку А. Аналіз результатів трьох алгоритмів показує чітку залежність ефективності моделей від якості попередньої обробки даних. На базовому сценарії А найслабший результат продемонструвала Логістична регресія з показниками точності 0.8381, Precision 0.8398, повноти 0.8381, F1-міри 0.8388 та значенням ROC-AUC 0.9165, тоді як Випадковий ліс досяг точності 0.9283, Precision 0.9284 та ROC-AUC 0.9300, а Градієнтний бустинг показав стабільний результат у 0.9300 для всіх п'яти метрик. Перехід до сценарію Б з повноцінним очищенням, інженерією ознак та масштабуванням дав потужний поштовх саме для Логістичної регресії, підвищивши її точність, Precision, Recall та F1-міру до рівних 0.9065, а ROC-AUC до 0.9290, у той час як показники Випадкового лісу з точністю 0.9283 та Градієнтного бустингу з точністю 0.9300 залишилися незмінними. Застосування аугментації SMOTE у сценарії В виявило різний вплив на алгоритми: Логістична регресія трохи погіршила свої результати до точності 0.8983, Precision 0.9009, F1-міри 0.8988 та ROC-AUC 0.9288 через утворення шуму в зонах перекриття класів, Випадковий ліс знову зберіг стабільність із точністю 0.9283 та Precision 0.9284, а Градієнтний бустинг продемонстрував найкращий результат серед усіх випробувань, піднявши точність, Precision, Recall, F1-Score та ROC-AUC до абсолютного максимуму в 0.9500 завдяки ефективній оптимізації розпізнавання міноритарних класів.

Таблиця 2.1. Порівняння результатів навчання моделей з метриками Accuracy Precision, Recall, F1-Score, ROC-AUC

Сценарій	Модель	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Сценарій А (Базовий)	Logistic Regression	0.8381	0.8398	0.8381	0.8388	0.9165
Сценарій А (Базовий)	Random Forest	0.9283	0.9284	0.9283	0.9283	0.93
Сценарій А (Базовий)	Gradient Boosting	0.93	0.93	0.93	0.93	0.93
Сценарій Б (Очищені дані)	Logistic Regression	0.9065	0.9065	0.9065	0.9065	0.929
Сценарій Б (Очищені дані)	Random Forest	0.9283	0.9284	0.9283	0.9283	0.93
Сценарій Б (Очищені дані)	Gradient Boosting	0.93	0.93	0.93	0.93	0.93
Сценарій В (Очищені + SMOTE)	Logistic Regression	0.8983	0.9009	0.8983	0.8988	0.9288
Сценарій В (Очищені + SMOTE)	Random Forest	0.9283	0.9284	0.9283	0.9283	0.93
Сценарій В (Очищені + SMOTE)	Gradient Boosting	0.95	0.95	0.95	0.95	0.95

1. Logistic Regression

Сильні сторони: Швидка у навчанні, легка у інтерпретації та схильна до суттєвого покращення після нормалізації даних (в Сценарії Б Accuracy зросла з 0.8381 до 0.9065).

Слабкі сторони: Чутлива до мультиколінеарності та нелінійних зв'язків. Показує найнижчу точність серед трьох алгоритмів, оскільки лінійній межі важко розділити складні багатовимірні класи сну.

2. Random Forest

Сильні сторони: Стійкий до відсутності масштабування, нормальності розподілу та наявності шумів. Демонструє стабільно високу точність (0.9283) вже на базових даних.

Слабкі сторони: Мало реагує на попередню обробку даних та SMOTE (метрики залишилися незмінними). Може мати високе споживання пам'яті при великій кількості дерев.

3. Gradient Boosting

Сильні сторони: Найкраща модель у дослідженні. Здатна послідовно виправляти помилки попередніх дерев, що дозволяє досягати найвищої точності (0.9500 у Сценарії B).

Слабкі сторони: Вимагає більше часу на тренування, чутлива до вибору гіперпараметрів та схильна до перенавчання за наявності сильного шуму у даних.

2.3 Аналіз впливу SMOTE та ризиків перенавчання

Чи покращила аугментація якість передбачення? Для Gradient Boosting так. Застосування SMOTE дало відчутний приріст усіх метриках з 0.9300 до 0.9500. Це пояснюється тим, що при початковому дисбалансі бустинг фокусувався переважно на переважному класі Moderate Debt (52.5%). Синтетичне врівноваження вибірки змусило алгоритм однаково якісно оптимізувати розділення міноритарних класів (зокрема Severe Sleep Debt 7.6%).

Для Logistic Regression — ні. Метрика Accuracy знизилася з 0.9065 до 0.8983. Оскільки SMOTE генерує нові точки вздовж лінії між найближчими сусідами, це може створювати синтетичні об'єкти в зонах перекриття класів, що ускладнює побудову чіткої лінійної розділювальної гіперплощини.

Для Random Forest — ні. Результати залишилися на рівні 0.9283, що свідчить про достатню вроджену здатність ансамблю дерев обробляти вихідний дисбаланс класів за рахунок випадкового вибору ознак та об'єктів.

За яких умов SMOTE викликає перенавчання? Якщо SMOTE застосовується до всього датасету до його розбиття на train/test, тестові дані потрапляють у процес генерації синтетичних сусідів. Це створює ілюзію ідеальної точності, яка насправді є наслідком перенавчання. Якщо міноритарний клас містить викиди, SMOTE створює навколо них нові синтетичні точки. Це змушує модель будувати занадто складні межі під шумні дані замість загальної закономірності. Якщо реальних прикладів класу занадто

мало, генерація нових екземплярів призводить до «заучування» вузького локального кластера, що погіршує узагальнюючу здатність моделі на нових реальних даних.

4. Висновок

У ході виконання лабораторної роботи було проведено порівняльний аналіз трьох алгоритмів машинного навчання на різних етапах підготовки даних. Результати демонструють, що якісна попередня обробка та трансформація ознак мають вирішальне значення для лінійних моделей, адже точність Логістичної регресії зросла з 0.8381 до 0.9065. Ансамблеві алгоритми виявилися значно стійкішими до початкового стану даних, а найкращий підсумковий результат показав градієнтний бустинг, досягнувши точності 0.9500 у сценарії з балансуванням класів.

Застосування методу SMOTE виявилося найефективнішим саме для бустингу, оскільки дозволило алгоритму краще розпізнавати міноритарні категорії боргу сну, тоді як для лінійної моделі воно створило незначний шум у зонах перекриття. Отже, оптимальною стратегією для розв'язання даної задачі класифікації є поєднання просунутого очищення даних, інженерії ознак, аугментації навчальної вибірки та використання алгоритму Gradient Boosting.

4. Список використаних джерел

1. Sleep Debt & Screen Time: Late Night Phone Habits [Електронний ресурс] : датасет / Samar Talwar // Kaggle. — Режим доступу: https://www.kaggle.com/datasets/samartalwar/sleep-debt-and-screen-time-late-night-phone-habits (дата звернення: 11.10.2026).

Додаток А

Lab 2

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, PowerTransformer, StandardScaler
from sklearn.impute import SimpleImputer, IterativeImputer
from imblearn.over_sampling import SMOTE

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.metrics import accuracy_score, precision_recall_fscore_support, roc_auc_score

import warnings
warnings.filterwarnings('ignore')
```

```
raw_df = pd.read_csv('data/bedtime_screentime_sleep_debt.csv')
target_col = 'sleep_debt_category'

le = LabelEncoder()
y = le.fit_transform(raw_df[target_col])
X = raw_df.drop(columns=['user_id', target_col], errors='ignore')

X_train_raw, X_test_raw, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

СЦЕНАРІЙ А: Базові дані (мінімальне очищення та One-Hot Encoding)

```
X_tr_A = pd.get_dummies(X_train_raw, drop_first=True)
X_te_A = pd.get_dummies(X_test_raw, drop_first=True)
X_tr_A, X_te_A = X_tr_A.align(X_te_A, join='left', axis=1, fill_value=0)

imputer_a = SimpleImputer(strategy='median')
X_train_A = imputer_a.fit_transform(X_tr_A)
X_test_A = imputer_a.transform(X_te_A)
```

СЦЕНАРІЙ Б: Повноцінно очищені дані (Імп'ютація + Feature Engineering + PowerTransformer)

```
X_tr_B_base = pd.get_dummies(X_train_raw, dummy_na=True)
X_te_B_base = pd.get_dummies(X_test_raw, dummy_na=True)
X_tr_B_base, X_te_B_base = X_tr_B_base.align(X_te_B_base, join='left', axis=1, fill_value=0)

# Імп'ютація
iter_imputer = IterativeImputer(random_state=42, max_iter=10)
X_train_B_imp = pd.DataFrame(iter_imputer.fit_transform(X_tr_B_base), columns=X_tr_B_base.columns)
X_test_B_imp = pd.DataFrame(iter_imputer.transform(X_te_B_base), columns=X_te_B_base.columns)

# Feature Engineering
for df_temp in [X_train_B_imp, X_test_B_imp]:
    if 'bedtime_phone_minutes' in df_temp.columns and 'screen_brightness_pct' in df_temp.columns:
        df_temp['screen_stimulation_load'] = df_temp['bedtime_phone_minutes'] * df_temp['screen_brightness_pct']

# Трансформація та масштабування
power_scaler = PowerTransformer(method='yeo-johnson', standardize=True)
X_train_B = power_scaler.fit_transform(X_train_B_imp)
X_test_B = power_scaler.transform(X_test_B_imp)
```

СЦЕНАРІЙ В: Очищені дані + Аугментація (SMOTE)

```
smote = SMOTE(random_state=42)
X_train_C, y_train_C = smote.fit_resample(X_train_B, y_train)
X_test_C, y_test_C = X_test_B, y_test
```

НАВЧАННЯ МОДЕЛЕЙ

```
models = {
    'Logistic Regression': LogisticRegression(max_iter=1000, random_state=42),
    'Random Forest': RandomForestClassifier(n_estimators=100, random_state=42),
    'Gradient Boosting': GradientBoostingClassifier(n_estimators=100, random_state=42)
}

scenarios = {
    'Сценарій А (Базовий)': (X_train_A, X_test_A, y_train, y_test),
    'Сценарій Б (Очищені дані)': (X_train_B, X_test_B, y_train, y_test),
    'Сценарій В (Очищені + SMOTE on Train)': (X_train_C, X_test_C, y_train_C, y_test_C)
}
```

```
results = []

for scenario_name, (X_tr, X_te, y_tr, y_te) in scenarios.items():
    for model_name, model in models.items():
        model.fit(X_tr, y_tr)
        y_pred = model.predict(X_te)

        acc = accuracy_score(y_te, y_pred)
        prec, rec, f1, _ = precision_recall_fscore_support(y_te, y_pred, average='weighted')

        try:
            y_proba = model.predict_proba(X_te)
            roc_auc = roc_auc_score(y_te, y_proba, multi_class='ovr', average='weighted')
        except AttributeError:
            roc_auc = np.nan

        results.append({
            'Сценарій': scenario_name,
            'Модель': model_name,
            'Accuracy': round(acc, 4),
            'Precision': round(prec, 4),
            'Recall': round(rec, 4),
            'F1-Score': round(f1, 4),
            'ROC-AUC': round(roc_auc, 4)
        })
```

```
results_df = pd.DataFrame(results)
results_df
```